

FORMAL LANGUAGES AND COMPILERS

prof.s Giovanni Agosta, Luca Breveglieri and Daniele Cattaneo

Exam of WEDNESDAY 12 FEBRUARY 2025 – Theory Part

LAST NAME (SURNAME) + FIRST NAME:

(capital letters please)

MATRICOLA:

(or PERSON CODE)

SIGNATURE:

TEACHER: ☐ Prof. G. AGOSTA – ☐ Prof. L. BREVEGLIERI – ☐ Prof. D. CATTANEO

INSTRUCTIONS – READ CAREFULLY:

- The exam is open book: textbooks and personal notes are permitted.
- Pencil writing is permitted, except on the cover page (this sheet).
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: lab part 1 h – theory part 2 h.

SCORING AND GRADE ATTRIBUTION:

- The exam is in written form and consists of two parts:

Theory syntax and semantics of languages, divided in four sections: (75% of the final grade)

- 1) regular expressions and finite automata (1 exercise × 10 points)
- 2) syntax analysis and parsing methodologies (1 exercise × 10 points)
- 3) grammar design – translation – semantic analysis (2 exercises × 5 points)
- 4) *bonus* question (1 exercise × 5 points)

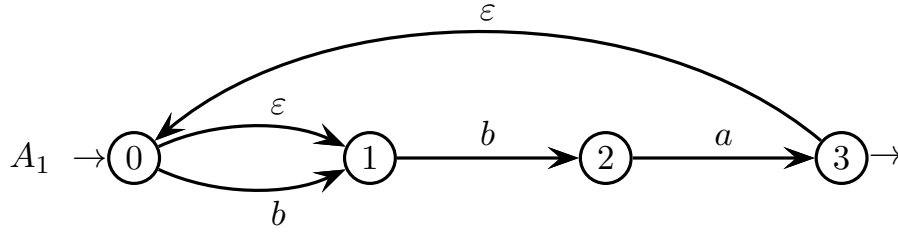
Lab compiler design by Flex and Bison (25% of the final grade)

- 1) basic questions (30 points)
- 2) *bonus* question (5 points)

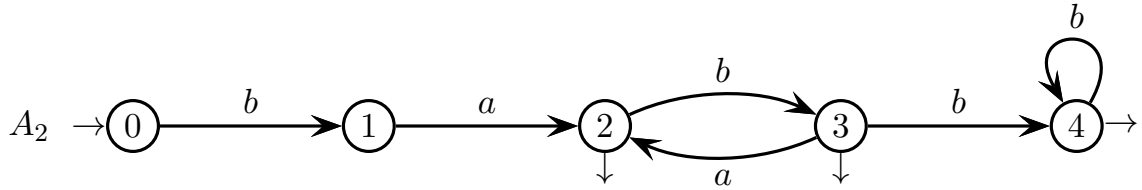
- For the theory part to be valid, the candidate must achieve a grade of at least 5/10 in each of the three sections 1, 2 and 3. If this is not the case, section 4 is not evaluated. Section 4 has no threshold on its own. Correctly answering all the questions in sections 1, 2 and 3 allows the candidate to achieve a grade of 30/30 (but not *cum laude*) for the theory part.
- For the lab part to be valid, the candidate must achieve a grade of at least 15/30 in the basic questions. The bonus question is evaluated only if the threshold is reached.
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- The final grade is the weighted average of the theory part (75 %) and of the lab part (25 %), and must be $\geq 18/30$ (*cum laude* $\geq 32/30$).

1 Regular expressions and finite automata (10 points)

1. Consider the nondeterministic automaton A_1 below, over the two-letter alphabet $\Sigma = \{a, b\}$:



the automaton deterministic A_2 below, over the same alphabet $\Sigma = \{a, b\}$:



and the regular expression R , also over the alphabet $\Sigma = \{a, b\}$:

$$R = ab^+ (ab)^* \mid (ba)^+ (ab)^+$$

Answer the following questions (use the figures / tables / spaces on the next pages):

- Apply the backward propagation method on automaton A_1 to produce an automaton A'_1 without epsilon-moves (spontaneous moves).
- Say if the automaton A'_1 , obtained at point (a) by eliminating the epsilon-moves, is deterministic or not. Briefly explain your answer.
- Use the Brzozowski-McCluskey node elimination method (BMC) to produce a regular expression R_{A_2} from automaton A_2 .
- Apply the Berry-Sethi (BS) method to the regular expression R to obtain an equivalent deterministic automaton A_R .
- Say if the automaton A_R obtained through the BS method at point (d) is minimal, and explain your answer. If the automaton is not minimal, minimize it.
- Say if the language described by the intersection $L(A_2) \cap L(R)$ is empty or not. Briefly explain your answer.

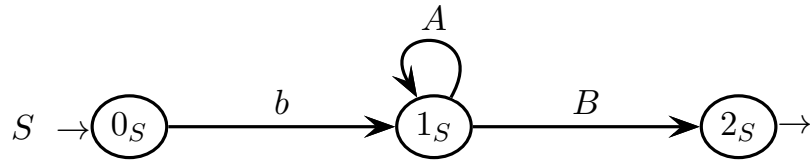
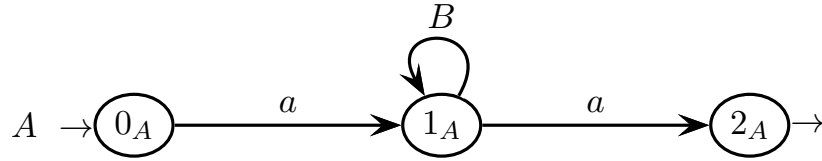
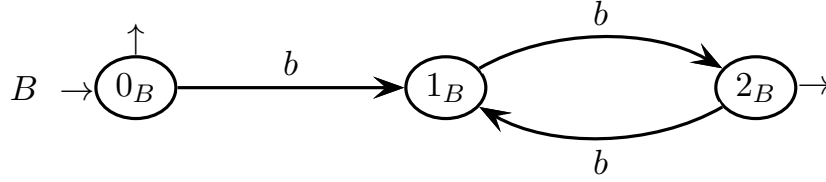
space for questions (a-f)

space for questions (a-f)

space for questions (a-f)

2 Syntax analysis and parsing methodologies (10 points)

1. Consider the following grammar G , described by a machine net, over the two-letter terminal alphabet $\Sigma = \{a, b\}$ and the three-symbol nonterminal alphabet $V = \{A, B, S\}$ (axiom S):

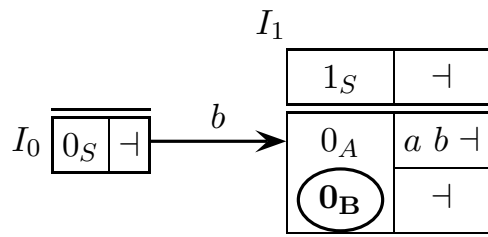


Answer the following questions (use the figures / tables / spaces on the next pages):

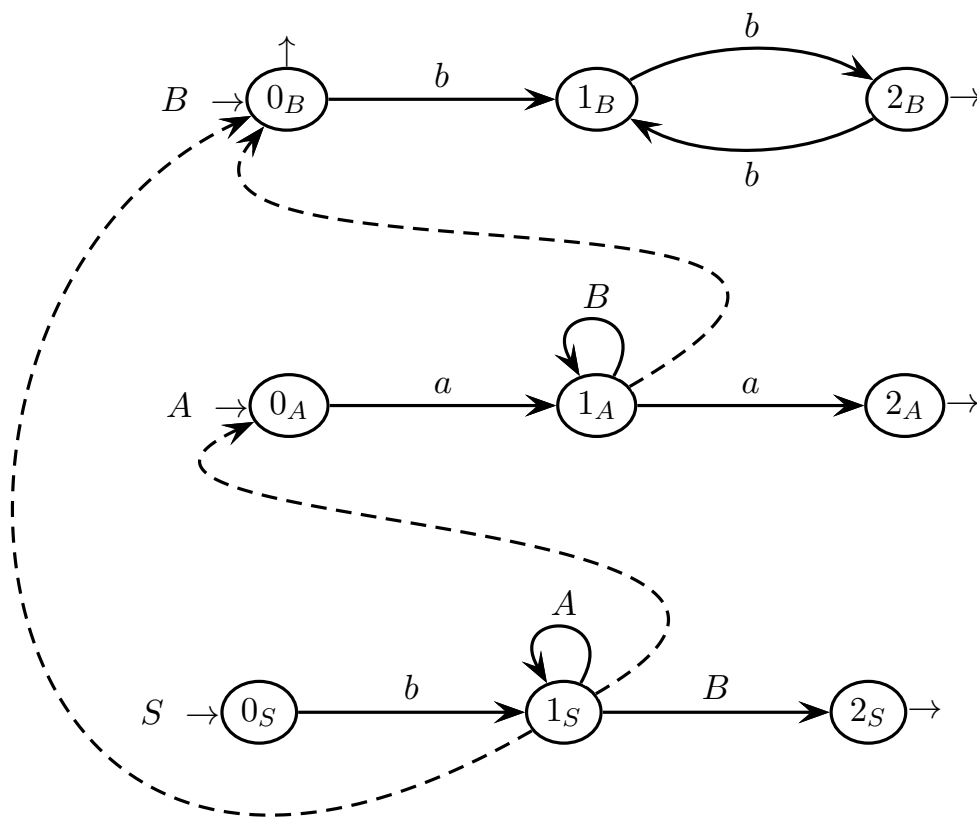
- List the strings of length 3 or less belonging to language $L(G)$ and draw their syntax trees.
- Build the complete pilot automaton and determine whether grammar G admits an ELR parser construction.
- Determine the guide sets on the machine net of grammar G and say whether there are conflicts that prevent the ELL parser construction.
- Independently of whether the machine net is ELL or not, write the recursive-descent parser procedure for the axiomatic machine S .
- Can you propose a modification to the machine net of grammar G that removes the conflicts without changing the language $L(G)$?

valid strings of length ≤ 3 and syntax trees – question (a)

pilot and conflict analysis – question (b)



guide sets and conflict analysis – question (c)



recursive-descent procedure for nonterminal S – question (d)

space for question (e)

3 Grammar design – translation – semantic analysis (5 + 5 points)

1. Consider a language that describes linear combinations of vectors, with the following features:
 - Scalars and vectors are denoted by terminals `scal` and `vect`, respectively. Both terminals are unexpanded.
 - Scalars can be added, with operator `+`, and the result is of scalar type; the associativity of scalar addition is unspecified.
 - A linear combination consists of the addition of vectors, where each vector may be multiplied by a scalar value. The result of a linear combination is of vector type.
 - The vector addition operator is `+`, and it is right-associative.
 - The operator for scalar-vector multiplication is `*`, and the result is of vector type. The scalar value must be the left factor, and it may appear as a single scalar or a sum of scalars (between parentheses).Scalar-vector multiplication has higher precedence on both scalar addition and vector addition.
 - *Caution*: scalar-to-vector (or vector-to-scalar) addition is disallowed.
 - Vector subexpressions are allowed, at any depth, enclosed between parentheses.
 - A language phrase is a linear combination, as described above.

Sample linear combinations (symbols `sn` and `Vn` denote scalars and vectors, respectively):

`V1`
`V1 + V2`
`V1 + (V2 + V3)`
`V1 + s2 * V2`
`V1 + s1 * (V2 + (s2 + s3) * V3)`

Answer the following questions (use the figures / tables / spaces on the next pages):

- (a) Write an EBNF non-ambiguous grammar (axiom `LComb`) that generates a language phrase.
- (b) Draw the syntax tree of the sample phrase:

`V1 + s1 * (V2 + (s2 + s3) * V3)`

- (c) Suppose now that scalars can be multiplied, with operator `*`; the result is of scalar type; scalar multiplication is *left-associative* and has higher precedence on scalar addition. Here is an example:

`V1 + (s1 + s2 * s3) * V2`

Modify the grammar of point (a), still EBNF and non-ambiguous, to include this generalization of the language.

2. Consider the language of the nested lists, possibly empty, surrounded by open and closed parentheses, with elements a and b , as modeled by the source BNF grammar G^s below (axiom S):

$$G^s \left\{ \begin{array}{l} 1: S \rightarrow (L) \\ 2: L \rightarrow a L \\ 3: L \rightarrow b L \\ 4: L \rightarrow S L \\ 5: L \rightarrow \varepsilon \end{array} \right.$$

Answer the following questions (use the tables / trees / spaces on the next pages):

- (a) Consider a syntactic translation τ that, in each (sub)list, accumulates all the elements b after the elements a . For example:

$$\tau((a b b (b a) a)) = (a (a b) a b b)$$

Write a translation grammar G^τ (or a scheme) that realizes translation τ , with source grammar G^s . Draw the translation tree (or the separate source and target trees) of the sample translation τ below:

$$\tau((a b (b a) a)) = (a (a b) a b)$$

- (b) Is translation τ deterministic ELL ? Explain your answer.
- (c) Is the inverse translation τ^{-1} deterministic ELL ? Is it deterministic in general, i.e., does a deterministic pushdown translator exist for it ? Explain your answers.
- (d) Can we define a syntactic translation (with source grammar G^s) that, in each (sub)list, accumulates all the sublists at the sub(list) end ? For example:

$$(a b (b) a ()) \quad \text{is translated to} \quad (a b a (b) ())$$

Explain your answer.

free space for continuation or notes – any questions

4 Bonus question (5 points)

1. Consider the following BNF grammar G (axiom S):

$$G \left\{ \begin{array}{l} S \rightarrow L \\ L \rightarrow A L \\ L \rightarrow \varepsilon \\ A \rightarrow a B \\ B \rightarrow b B \\ B \rightarrow \varepsilon \end{array} \right.$$

Note that it holds $L(G) = (ab^*)^*$, thus grammar G models a regular language.

We want to design an attribute grammar, based on the syntactic support G , to determine if a string of language $L(G)$ also belongs to the non-context-free language L' below:

$$L' = \{ (ab^n)^n \mid n \geq 0 \}$$

that is, if the number of letters b between each consecutive pair of letters a is the same as the number of letters a in the whole string. For example, string aa belongs to $L(G)$ but not to L' , whereas string $abba$ belongs to both languages, so it is acceptable.

The attribute grammar must produce a boolean attribute in the root node S of the syntax tree, which is true or false if the string is acceptable or not. We also want the attribute grammar to be suitable for integration in an ELL parser based on grammar G .

Answer the following questions (use the figures / tables / spaces on the next pages):

- (a) Define what attributes are needed for this purpose, what their types are (left/right, data type), and to which nonterminals they are associated. One of the attributes is already given.
 - (b) Write the semantic functions for all the attributes defined at point (a).
 - (c) Prove that the devised attribute grammar has the one-sweep and L properties.
 - (d) Implement the corresponding integrated recursive-descent syntactic-semantic parsing procedure for nonterminal L .
-

list of attributes to be defined (the number of rows is not significant) – question (a)

<i>left/right</i>	<i>name</i>	<i>data type</i>	<i>nonterminal(s)</i>	<i>meaning</i>
left	r	boolean	S	correctness predicate

semantic functions – question (b)

#	<i>syntax</i>	<i>semantics</i>
1:	$S_0 \rightarrow L_1$	
2:	$L_0 \rightarrow A_1 L_2$	
3:	$L_0 \rightarrow \varepsilon$	
4:	$A_0 \rightarrow a B_1$	
5:	$B_0 \rightarrow b B_1$	
6:	$B_0 \rightarrow \varepsilon$	

space for questions (c-d)

free space for continuation or notes – any questions